

Lab02_tavares

Arthur Tavares - 237053

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Resposta: As estatísticas suficientes são o total de voos válidos e o total de voos atrasados.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
 - e. A função deve receber apenas dois argumentos:
 - i. `input`: o conjunto de dados (referente ao lote em questão);
 - ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. **É importante** observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

```
filter, lag
```

Os seguintes objetos são mascarados por 'package:base':

`intersect, setdiff, setequal, union`

```
library(readr)

#i e ii
getStats <- function(input, pos){
  input |>
  #a
  filter(AIRLINE %in% c("AA", "DL", "UA", "US")) |>
  #b
  filter(!is.na(ARRIVAL_DELAY)) |>
  #c
  group_by(DAY, MONTH, YEAR, AIRLINE) |>
  #d
  summarise(
    ntotal = n(),
    natrasados = sum(ARRIVAL_DELAY > 10),
    .groups = "drop"
  )
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de *callback* para instanciar DataFrames aplicando a função `getStats` criada acima;
 - c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
colunas <- cols_only(
  YEAR = "integer",
  MONTH = "integer",
  DAY = "integer",
  AIRLINE = "character",
  ARRIVAL_DELAY = "double"
)

lotes <- read_csv_chunked(
```

```

file = "flights.csv.zip",
#a
chunk_size = 100000,
#b
callback = DataFrameCallback$new(getStats),
#c
col_types = columnas
)

```

4. Crie uma função chamada `computeStats` que:

- a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b. Retorne as informações em um `tibble` contendo apenas as seguintes colunas:
 - i. Cia: sigla da companhia aérea;
 - ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 - iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1].

```

computeStats <- function(dados) {
  dados |>
    #a
    group_by(AIRLINE, YEAR, MONTH, DAY) |>
    summarise(
      somatotal = sum(ntotal),
      somatrasados = sum(natrasados),
      .group = "drop"
    ) |>
    #b
    mutate(
      #i
      Cia = AIRLINE,
      #ii
      Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")),
      #iii
      Perc = somatrasados/somatotal
    ) |>
    select(Cia, Data, Perc)
}

estatisticafinal <- computeStats(lotes)

```

```
`summarise()` has regrouped the output.
Adding missing grouping variables: `AIRLINE`, `YEAR`, `MONTH`
i Summaries were computed grouped by AIRLINE, YEAR, MONTH, and DAY.
i Output is grouped by AIRLINE, YEAR, and MONTH.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(AIRLINE, YEAR, MONTH, DAY))` for per-operation
  grouping (`?dplyr::dplyr_by`) instead.
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.
 - a. Instale e carregue os pacotes `ggcal` e `ggplot2`.
 - b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
 - c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (`tibble` com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
 - i. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 - ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 - iii. Retornar para o usuário a base do calendário criada acima.
 - d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
 - e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

```
#a
library(ggplot2)
library(devtools)
```

Carregando pacotes exigidos: `usethis`

```
library(ggcal)
#b
paletacores <- scale_fill_gradient(low = "#4575b4", high = "#d73027")
pal <- paletacores
#c
baseCalendario <- function(stats, cia) {
```

```

#i
subconjunto <- stats |>
  filter(Cia == cia)

#ii
basedocalendario <- ggcal(subconjunto$Data, subconjunto$Perc)

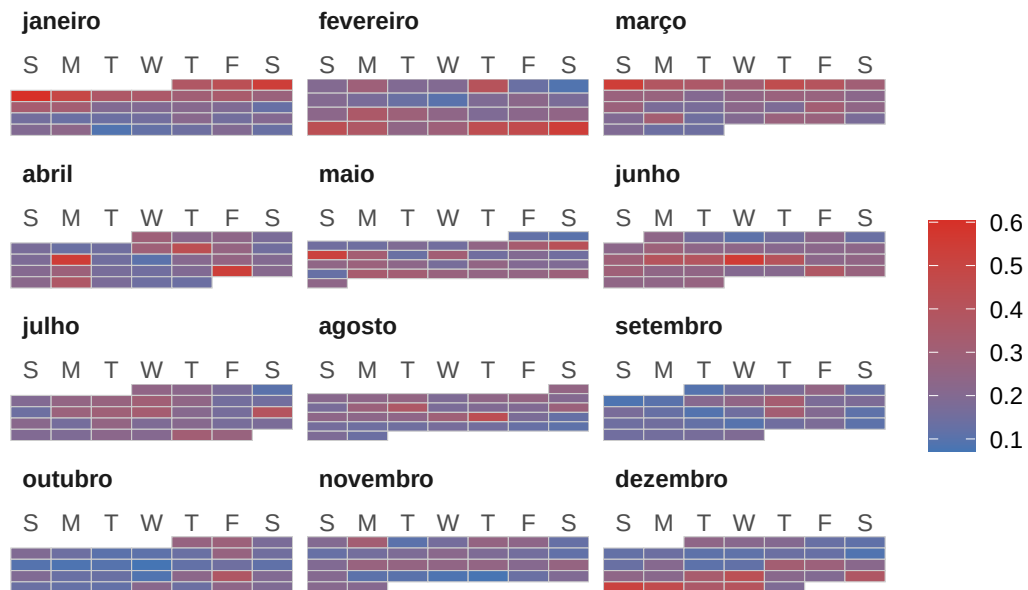
#iii
return(basedocalendario)
}

#d
cAA <- baseCalendario(estatisticafinal, "AA")
cDL <- baseCalendario(estatisticafinal, "DL")
cUA <- baseCalendario(estatisticafinal, "UA")
cUS <- baseCalendario(estatisticafinal, "US")

#e
cAA + pal + ggtitle("Atrasos na Chegada - American Airlines (AA)")

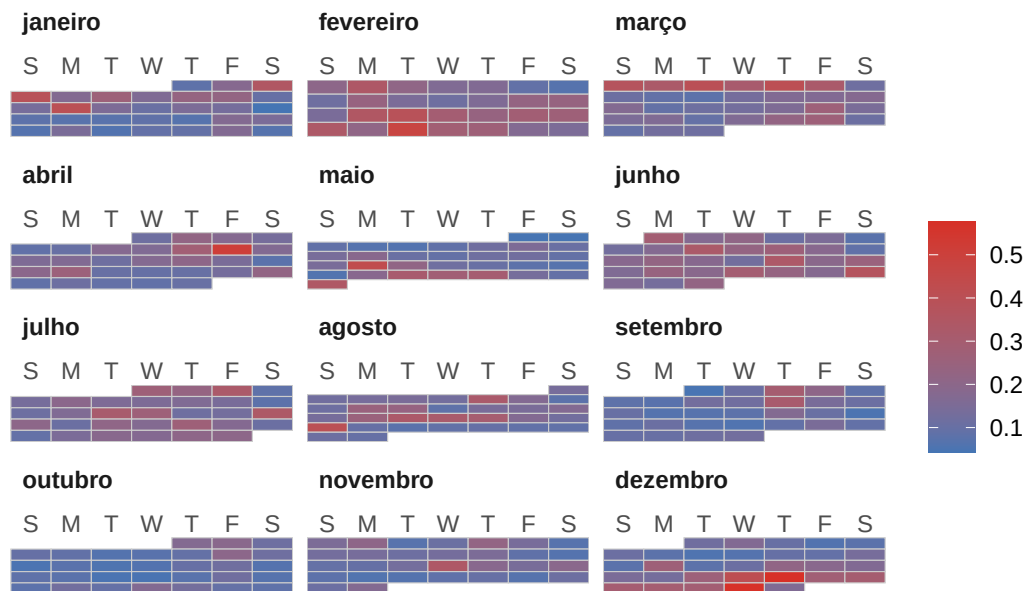
```

Atrasos na Chegada – American Airlines (AA)



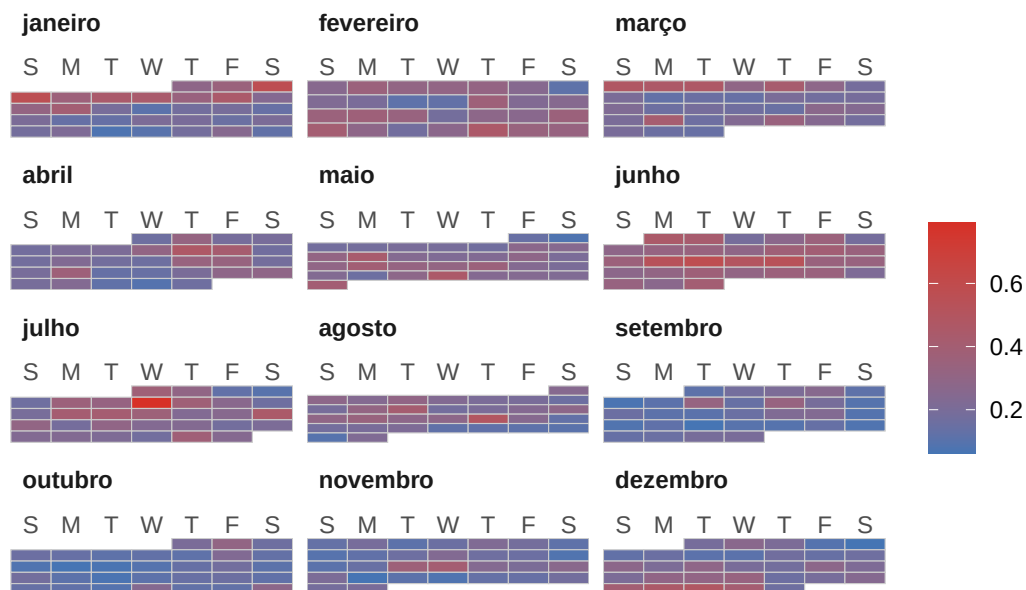
```
cDL + pal + ggtitle("Atrasos na Chegada - Delta Air Lines (DL)")
```

Atrasos na Chegada – Delta Air Lines (DL)



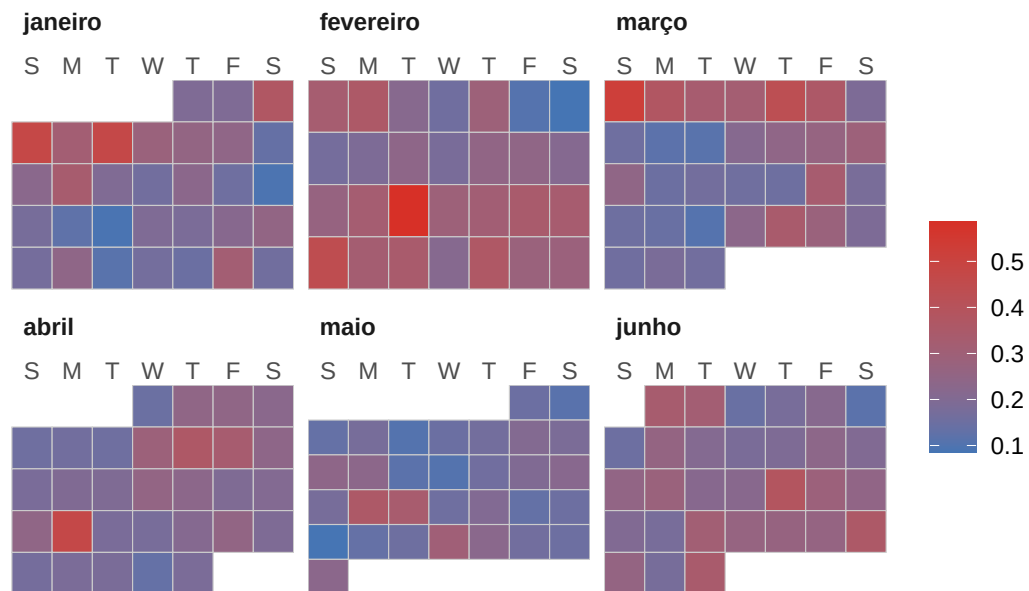
```
cUA + pal + ggtitle("Atrasos na Chegada - United Airlines (UA)")
```

Atrasos na Chegada – United Airlines (UA)



```
cUS + pal + ggtitle("Atrasos na Chegada - US Airways (US)")
```

Atrasos na Chegada - US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-24 18:46:29 -03"
```